SMART RESUME SCREENING TOOL

Under the Guidance Name : Dr.Anita Ma’am
Department of AI&DS
KLH University

Team Members:
2510080029-Sudeeptha
2510080037-Lokrey Latika
2510080043-Sai Siri
2510080061-Kovidha

INTRODUCTION

BACKGROUND

High volume. Limited attention.

Recruiters receive many resumes for every opening. Manual review consumes time, and important skills or qualifications can be missed.

THE NEED

Automate screening

Identify suitable candidates quickly

Reduce recruiter workload

Rank candidates by job relevance

PROBLEM STATEMENT

Develop a Java-based tool that analyzes resumes, compares candidate skills with job requirements, calculates matching scores, and ranks candidates by suitability.

MANUAL SCREENING DRAWBACKS

01

Time-consuming

02

Large resume volume

03

Repetitive work

04

Inconsistent comparison

OBJECTIVES

1. Extract candidate information
2. Identify skills & keywords
3. Compare with job descriptions
4. Handle spelling variations
5. Calculate relevance scores
6. Rank using Priority Queue / Heap
7. Reduce manual screening effort

LITERATURE REVIEW

EXISTING APPROACHES

- Manual resume screening
- Keyword-based filtering
- Applicant Tracking Systems
- Automated matching systems

LIMITATIONS

- Exact keyword matching may miss spelling variations.
- Basic systems may not provide effective ranking.
- Manual screening requires significant time.
- Some approaches lack efficient DSA-based ranking.

OUR FOCUS

Combine String Matching + Edit Distance + Priority Queue for efficient screening.

PROPOSED SYSTEM

The Smart Resume Screening Tool:

- Accepts candidate resumes.
- Reads and parses resume information.
- Reads job requirements.
- Matches resume content with job requirements.
- Calculates a relevance score.
- Ranks candidates.
- Displays the final ranking.

KEY FEATURES

Resume parsing

Keyword matching

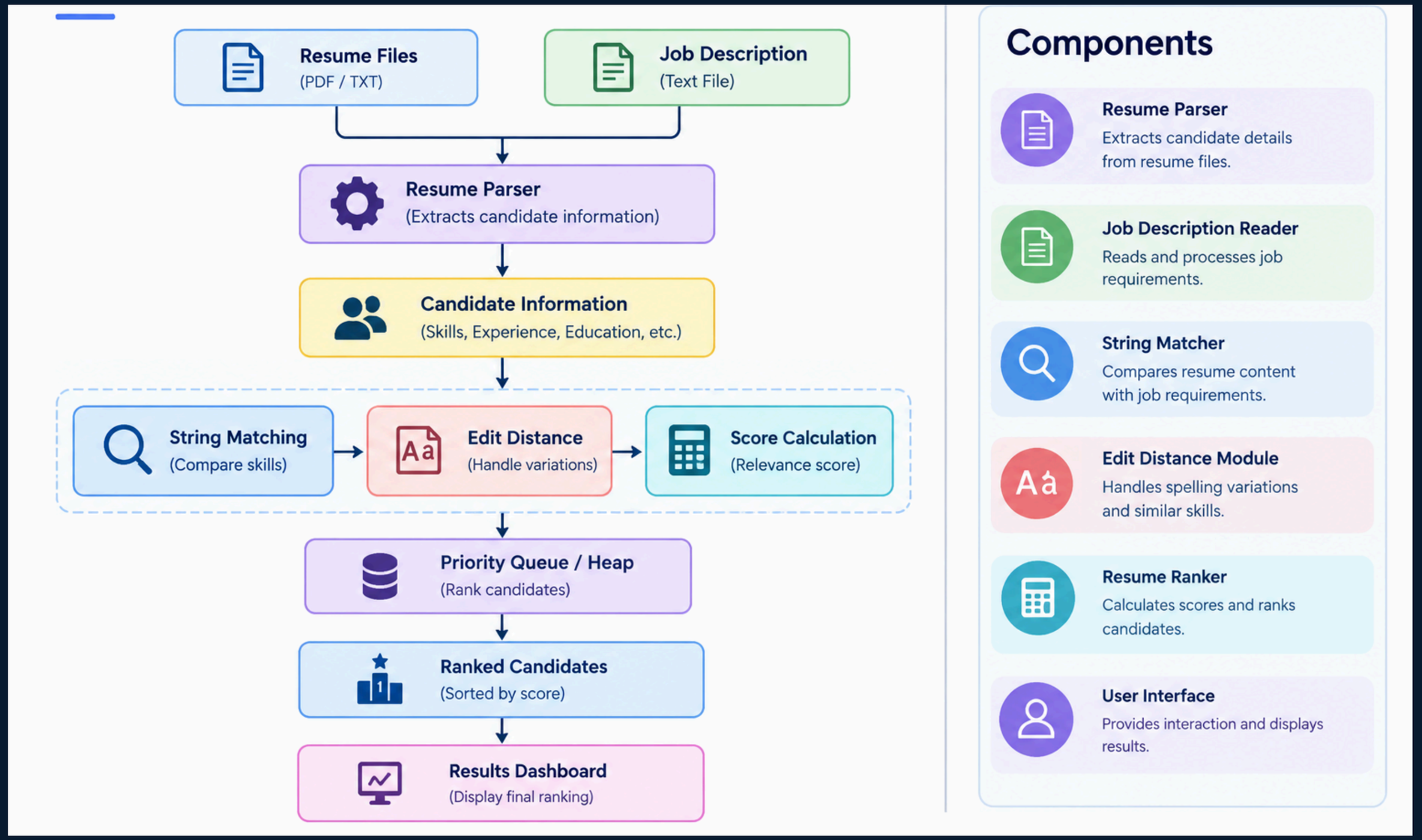
Fuzzy matching

Candidate scoring

Priority - based ranking

Simple User Interface

PROPOSED SYSTEM



METHODOLOGY

1 Upload / read resumes

2 Parse candidate information

3 Read job description

4 Extract relevant keywords

5 Perform string matching

6 Apply Edit Distance

7 Calculate candidate score

8 Insert into Priority Queue

9 Generate ranked results

Edit Distance uses Dynamic Programming • O(m × n) time complexity

TOOLS & TECHNOLOGIES

PROGRAMMING LANGUAGE

Java

ENVIRONMENT

Eclipse IDE / IntelliJ IDEA

DATA STRUCTURES

HashMap • Priority Queue • Arrays

ALGORITHMS

String Matching • Edit Distance • Searching • Sorting • Ranking

FILE HANDLING

Java File I/O

DATABASE

MySQL

IMPLEMENTATION

Resume Parser

Reads resume files and extracts candidate name and content.

Job Description

Reads required job skills.

String Matching

Compares resume content with job requirements.

Edit Distance

Handles spelling mistakes and variations.

Resume Ranker

Calculates scores and ranks with a Priority Queue.

Main Module

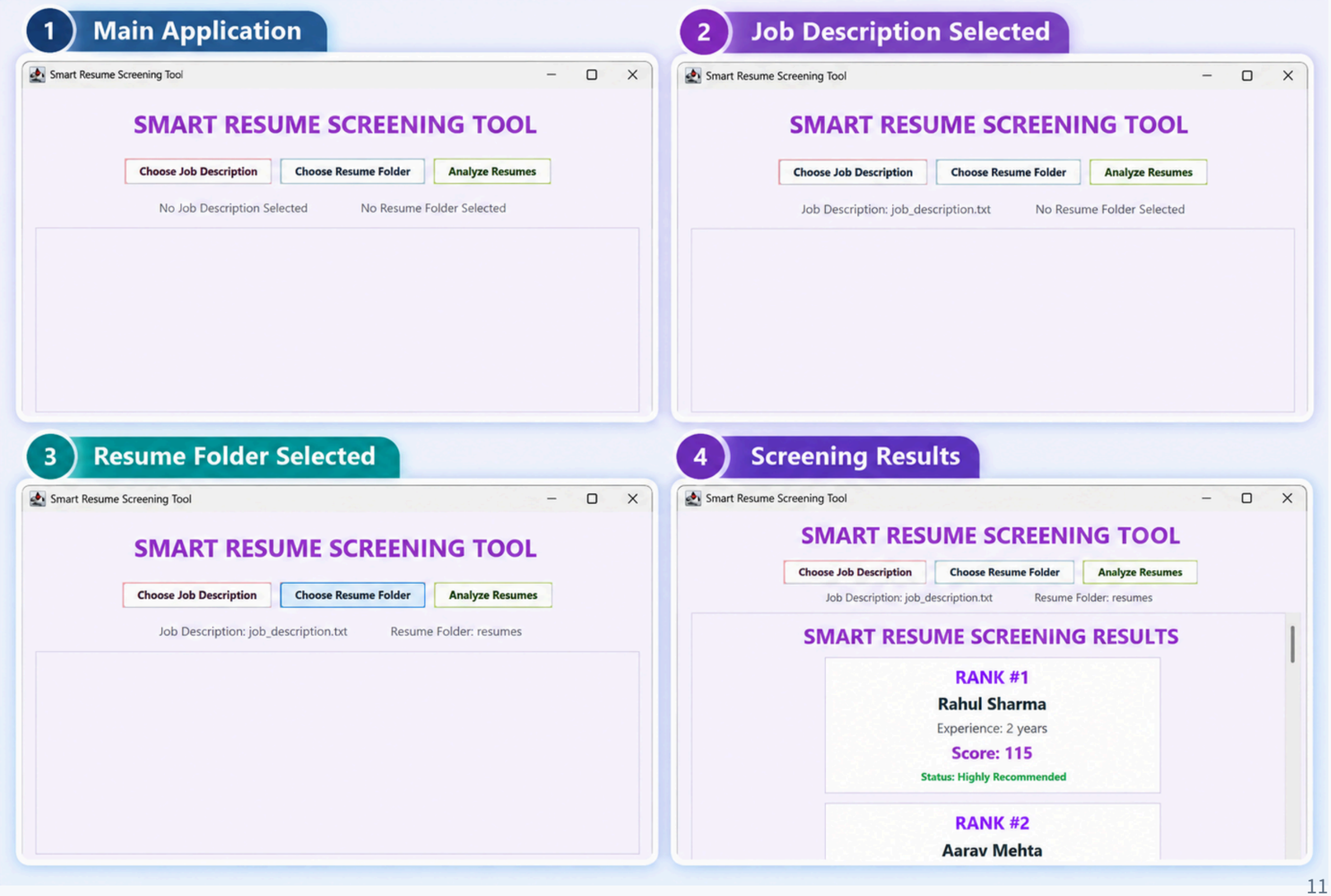
Controls the complete workflow.

Modular Java structure: separate class files plus Main.java

RESULTS & SCREENSHOTS

SYSTEM RESULTS

- Reads multiple resumes.
- Extracts candidate information.
- Matches job requirements.
- Calculates scores.
- Ranks and displays candidates.
- Displays screening results.



TESTING

Test Case	Input	Expected Result
TC01	Valid resume	Resume information extracted
TC02	Job description	Requirements loaded
TC03	Matching skills	Score increases
TC04	Spelling variation	Edit Distance detects similarity
TC05	Multiple candidates	Candidates ranked
TC06	Empty/invalid file	Error handled

Testing Result:

The major modules were tested individually and then integrated into the complete screening workflow.

ADVANTAGES

01

Saves recruiter time

02

Reduces manual effort

03

Processes multiple resumes

04

Provides candidate ranking

05

Handles spelling variations

06

Demonstrates efficient DSA

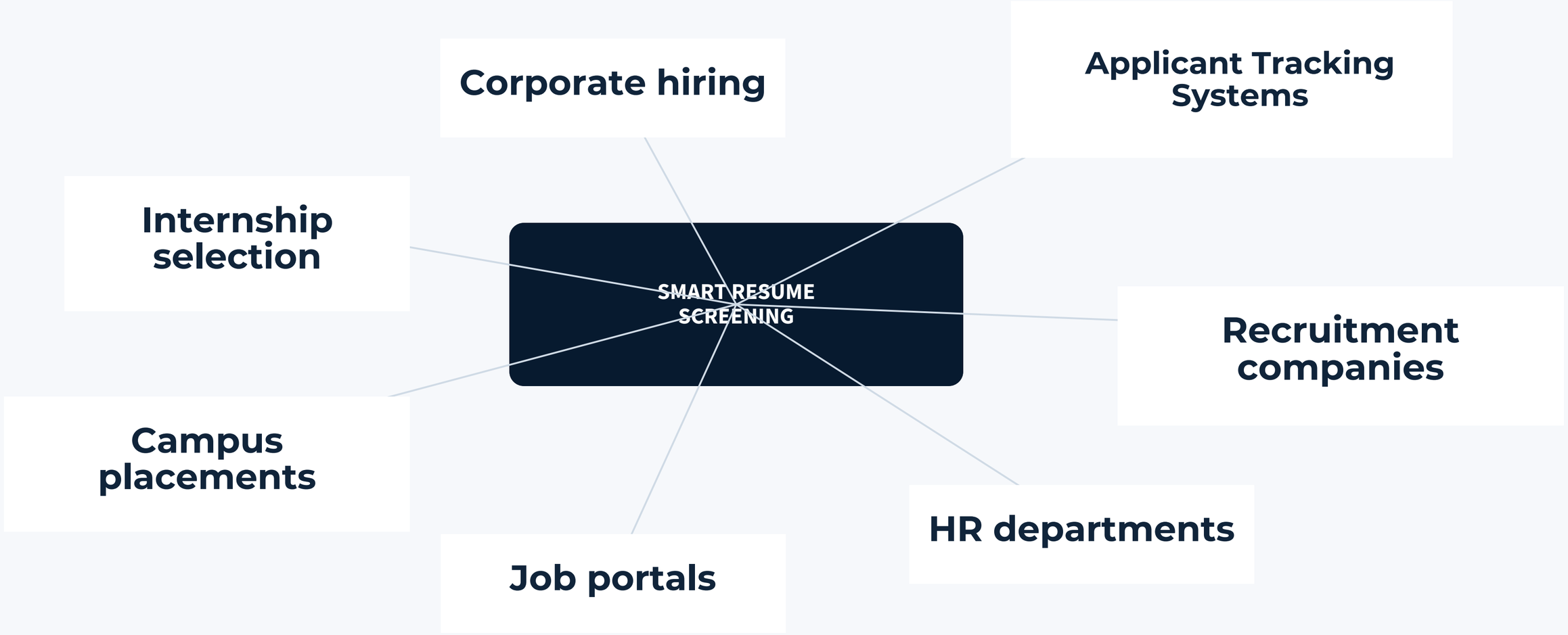
07

Easy to understand & extend

08

Consistent screening criteria

APPLICATIONS



CHALLENGES FACED

Different spellings of the same skill	—	**Edit Distance**
Ranking multiple candidates	—	**Priority Queue**
Reading resume files	—	**Java File I/O**
Matching candidate skills	—	**String Matching**
Organizing the project	—	**Separate classes / modules**

FUTURE ENHANCEMENTS

- PDF / DOCX support
- Advanced resume parsing
- Accurate skill extraction
- Experience-based scoring
- Education & certification scoring
- Database integration
- ML-based ranking
- Web recruiter dashboard
- Scale to more candidates

CONCLUSION

- Resumes are parsed automatically.
- Skills are compared with job requirements.
- Edit Distance handles minor variations,.
- Candidate scores are calculated.
- Priority Queue ranks candidates efficiently.

FINAL OUTCOME

The system reduces manual screening effort and helps recruiters identify suitable candidates more efficiently.

REFERENCES

01 Java Documentation

02 Data Structures and Algorithms textbooks

03 Research papers: resume screening

04 Research papers: string matching

05 Research papers: Edit Distance

06 Research papers: Applicant Tracking Systems

07 Relevant technical websites and documentation

THANK YOU.